

Lecture 5 Deep Neural Netowork

Huichen Zhu



香港中文大學
The Chinese University of Hong Kong



Table of Contents

- Limitations of Fixed Basis Functions
- Multilayer Networks
- Deep Networks
- Error Functions



Table of Contents

- Limitations of Fixed Basis Functions
- Multilayer Networks
- Deep Networks
- Error Functions



Linear models with nonlinear basis functions

$$y(\mathbf{x}, \mathbf{w}) = f\left(w_0 + \sum_{j=1}^{M} w_j \phi_j(\mathbf{x}) \right).$$

- The basis functions $\phi_j(\mathbf{x})$ transform the input into features.
- We learn the coefficients w_j from the training data.
- For regression, $f(a) = a$. For binary classification, we can use a sigmoid.

Example: $\phi_1(x) = x$, $\phi_2(x) = x^2$, and $\phi_3(x) = x^3$.

The limitation of fixed features

A sufficiently rich set of basis functions can approximate a broad class of functions.

- But which features should we choose before seeing the data?
- How many features do we need when the input has many coordinates?

$$\mathbf{x} \in \mathbb{R}^D, \qquad \boldsymbol{\phi}(\mathbf{x}) = (\phi_1(\mathbf{x}), \ldots, \phi_M(\mathbf{x}))^\top.$$

Central difficulty

Generic features that cover the whole input space can become prohibitively numerous as D increases.

Polynomial features in many dimensions

For one input, a cubic polynomial has four coefficients:

$$y(x) = w_0 + w_1 x + w_2 x^2 + w_3 x^3.$$

For D inputs, we also need interactions such as

$$x_i x_j, \qquad x_i x_j x_k.$$

The number of distinct monomials of total degree at most M is

$$\boxed{\binom{D+M}{M} = O(D^M) \quad \text{for fixed } M.}$$

For $D = 100$ and $M = 3$, this gives $\binom{103}{3} = 176{,}851$ coefficients.

Adding input variables rapidly expands the feature set.

A grid classifier and the curse of dimensionality

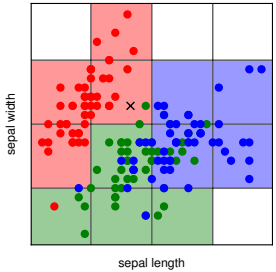


Figure 6.2: Grid classifier

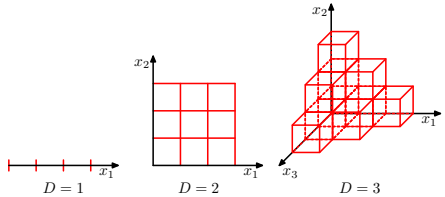


Figure 6.3: Grid cells in $D = 1, 2, 3$

Divide each coordinate into q intervals:

$$\boxed{\text{Number of cells} = q^D.}$$

D	1	2	10
Cells for $q = 10$	10	100	10^{10}

Predict the majority class in the test point's cell. With finite data, many cells become empty as D grows.

Volume near the boundary

The volume of a D-dimensional ball of radius r scales as

$$V_D(r) = K_D r^D.$$

For a unit ball, the fraction of volume in its outer shell of thickness ϵ is

$$\boxed{\frac{V_D(1) - V_D(1-\epsilon)}{V_D(1)} = 1 - (1-\epsilon)^D.}$$

D	2	3	10	100
Outer 10%	19%	27.1%	65.1%	99.997%

In high dimensions, nearly all the volume lies close to the boundary.



A Gaussian in One dimension

$$X \sim N(0, 1).$$

The density is highest at $X = 0$.
Distance from the centre:

$$R = |X|.$$

The density of R also peaks at zero:

$$p_R(r) = \sqrt{\frac{2}{\pi}} e^{-r^2/2}, \quad r \geq 0.$$

About 68% of observations lie within distance 1 of the centre.

A Gaussian in One dimension

$$X \sim N(0, 1).$$

The density is highest at $X = 0$.
Distance from the centre:

$$R = |X|.$$

The density of R also peaks at zero:

$$p_R(r) = \sqrt{\frac{2}{\pi}} e^{-r^2/2}, \quad r \geq 0.$$

About 68% of observations lie within distance 1 of the centre.



A Gaussian in high dimensions

Consider independent standard Gaussian coordinates:

$$\mathbf{X} = (X_1, \ldots, X_D)^\top, \qquad X_i \overset{\text{iid}}{\sim} N(0,1).$$

The joint density is

$$p(\mathbf{x}) = (2\pi)^{-D/2} \exp\left(-\frac{\|\mathbf{x}\|^2}{2}\right).$$

Its maximum is at $\mathbf{x} = \mathbf{0}$.

Now consider the distance of a random observation from the origin:

$$R = \|\mathbf{X}\| = \sqrt{X_1^2 + \cdots + X_D^2}.$$

Where does a typical observation lie?

Why the typical radius is approximately $\sqrt{D}$

The squared radius divided by D is an average:

$$\frac{R^2}{D} = \frac{1}{D}\sum_{i=1}^{D} X_i^2.$$

Since $E[X_i^2] = 1$, the law of large numbers gives

$$\frac{R^2}{D} \xrightarrow{P} 1, \qquad \boxed{\frac{R}{\sqrt{D}} \xrightarrow{P} 1.}$$

For example, when $D = 100$, a typical radius is approximately 10.

Density at a point and probability in a shell

All points at radius r have the same joint density, proportional to $e^{-r^2/2}$.

But the area of a sphere at radius r is proportional to r^{D-1}.

$$p_R(r) \propto \underbrace{r^{D-1}}_{\text{shell area}} \underbrace{e^{-r^2/2}}_{\text{density at each point}}, \quad r \geq 0.$$

For small Δr,

$$P(r \leq R \leq r + \Delta r) \approx p_R(r)\Delta r.$$

- $p(\mathbf{x})$ peaks at the origin.
- $p_R(r)$ peaks at $r = \sqrt{D-1}$ when $D > 1$.

Lower density over a much larger shell can give greater total probability.

What does a “thin shell” mean?

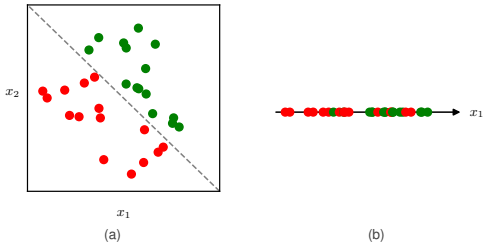
For large D, Gaussian observations concentrate around radius $\sqrt{D}$.

$$\frac{R}{\sqrt{D}} \approx 1.$$

The shell is thin *relative to its radius*.

- The typical radius grows with $\sqrt{D}$.
- The standard deviation of R approaches $1/\sqrt{2}$.
- The relative spread therefore becomes small.

Extra dimensions can provide useful information



- **(a) Observe both x_1 and x_2:** a straight line separates the classes.
- **(b) Observe only x_1:** the projected classes overlap.

Additional features can help

An informative extra coordinate can make classification easier. The curse of dimensionality does not mean every additional feature harms prediction.

Source: Bishop and Bishop, *Deep Learning* (2024), p. 177, Figure 6.6.



Data manifolds

A colour image with $H \times W$ pixels has $D = 3HW$ input values.

Yet realistic images occupy only a small, structured part of this space.

- Nearby pixels have strong dependencies.
- Random pixel values almost never produce a natural image.

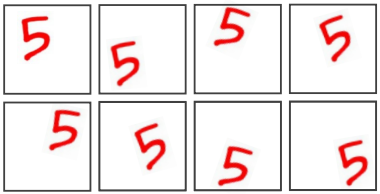
For a fixed object, changing only horizontal position, vertical position, and orientation gives an idealized representation

$$\mathbf{x} = g(u, v, \theta) \in \mathbb{R}^D.$$

The image varies along a manifold with intrinsic dimension approximately 3, despite a large ambient dimension D.

Data manifolds

Many pixels, few degrees of freedom

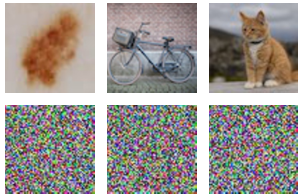


For a fixed digit shape, vary only horizontal position u, vertical position v, and rotation θ:

$$\mathbf{x} = g(u, v, \theta) \in \mathbb{R}^D.$$

The images approximately lie on a nonlinear manifold of intrinsic dimension 3.

Natural images occupy a restricted region



Top: natural images. Bottom: independent random pixel colours.

Both have $D = 64 \times 64 \times 3 = 12{,}288$ values, but natural images have strong dependencies between pixels.

Features can adapt to the region occupied by the data instead of covering the entire input space.



Basis functions adapted to the data

If observations concentrate on a manifold, features can focus on that region.

- Handcrafted features use domain knowledge to select useful transformations.
- RBF models can place basis centres at training inputs.
- Kernel SVMs retain a subset of training inputs as support vectors.

$$y(\mathbf{x}) = w_0 + \sum_j w_j \phi_j(\mathbf{x}).$$

The choice of ϕ_j now reflects the training data.

This provides an intermediate step toward learning the features themselves.

Radial basis function (RBF) models

A radial basis function depends on distance from a centre $\mathbf{c}_j$:

$$\phi_j(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{c}_j\|^2}{s^2}\right), \qquad s > 0.$$

- $\phi_j(\mathbf{c}_j) = 1$. The response decreases with distance.
- The width s controls how far the feature extends.

The chapter chooses one centre per training input, $\mathbf{c}_j = \mathbf{x}_j$:

$$y(\mathbf{x}) = w_0 + \sum_{n=1}^{N} w_n \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_n\|^2}{s^2}\right).$$

We learn the output coefficients. The centres follow the observed data rather than a uniform grid.

RBF models: benefits and limitations

For scalar regression, a regularized fitting objective is

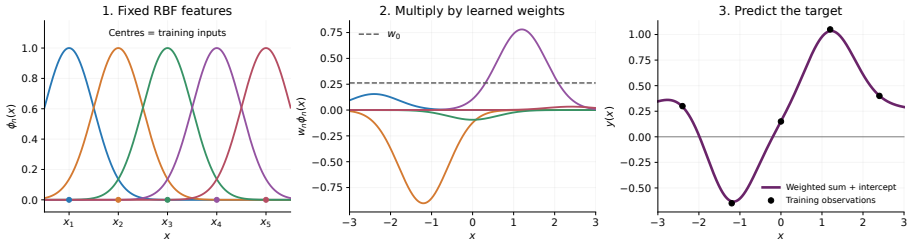
$$\min_{w_0,\mathbf{w}} \frac{1}{2}\sum_{n=1}^{N}\{y(\mathbf{x}_n) - t_n\}^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2.$$

- The centres adapt to where observations occur.
- In the one-centre-per-observation construction, the number of features grows with N.
- Narrow basis functions can overfit. Width selection and regularization matter.

RBF models can also use fewer centres selected by clustering. The chapter focuses on centres at all training inputs.

Radial basis functions as local features

$$\phi_n(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{x}_n\|^2}{s^2}\right), \qquad y(\mathbf{x}) = w_0 + \sum_{n=1}^{N} w_n \phi_n(\mathbf{x}).$$



- Each feature responds most strongly near its training-input centre.
- For fixed centres and width s, training learns $w_0, w_1, \ldots, w_N$.
- Positive or negative weights combine the local responses into a prediction curve.

Support vector machines (SVMs)

For binary labels $t_n \in \{-1, +1\}$, an SVM balances a wide margin against margin violations.

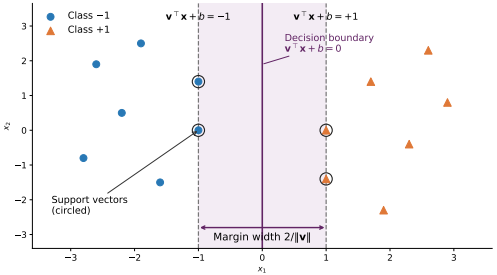
$$\min_{\mathbf{v}, b, \boldsymbol{\xi}} \frac{1}{2}\|\mathbf{v}\|^2 + C \sum_{n=1}^{N} \xi_n$$

subject to

$$t_n\{\mathbf{v}^\top \boldsymbol{\phi}(\mathbf{x}_n) + b\} \geq 1 - \xi_n,$$

$$\xi_n \geq 0.$$

- Smaller $\|\mathbf{v}\|$ favours a wider margin.
- ξ_n allows margin violations.
- C controls their penalty.



Circled points are support vectors. The figure shows a separable linear example, with $\phi(\mathbf{x}) = \mathbf{x}$ and no margin violations.



Table of Contents

- Limitations of Fixed Basis Functions
- Multilayer Networks
- Deep Networks
- Error Functions



Learning the features themselves

A single-layer basis-function model uses fixed features:

$$y(\mathbf{x}) = f\left(w_0 + \sum_{j=1}^{M} w_j \phi_j(\mathbf{x})\right).$$

Give each feature its own learnable parameters:

$$\phi_j(\mathbf{x}; \boldsymbol{\theta}_j) = h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + b_j^{(1)}\right).$$

- The first layer learns the feature transformations.
- The second layer learns how to combine the features into predictions.
- Training adjusts both layers together using a common objective.

The computation of one hidden unit

For hidden unit j, first form an affine combination of the inputs:

$$a_j^{(1)} = \sum_{i=1}^{D} w_{ji}^{(1)} x_i + b_j^{(1)}.$$

Then apply a nonlinear activation:

$$\boxed{z_j = h(a_j^{(1)}).}$$

Quantity	Meaning
$w_{ji}^{(1)}$	Weight from input i to hidden unit j
$b_j^{(1)}$	Bias of hidden unit j
$a_j^{(1)}$	Pre-activation
z_j	Hidden activation, or learned feature value



Combining hidden features into outputs

With M hidden units and K outputs, output unit k computes

$$a_k^{(2)} = \sum_{j=1}^{M} w_{kj}^{(2)} z_j + b_k^{(2)}, \qquad y_k = f(a_k^{(2)}).$$

Substituting the hidden activations gives

$$y_k = f\left[\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + b_j^{(1)}\right) + b_k^{(2)}\right].$$

- Hidden activation h shapes the learned features.
- Output activation f gives the required kind of prediction.
- The book calls this a two-layer network: two layers of learnable weights.

A forward pass with actual numbers

Take $\mathbf{x} = (1, 2)^\top$ and use ReLU hidden units:

$$\mathbf{W}^{(1)} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}, \qquad \mathbf{b}^{(1)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}.$$

The hidden computation gives

$$\mathbf{a}^{(1)} = \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \qquad \mathbf{z} = \max(\mathbf{0}, \mathbf{a}^{(1)}) = \begin{pmatrix} 0 \\ 2 \end{pmatrix}.$$

For an identity output with $\mathbf{W}^{(2)} = (2, -1)$ and $b^{(2)} = 0.5$,

$$y = 2(0) - 1(2) + 0.5 = -1.5.$$

ReLU acts on each hidden coordinate separately. Forward propagation evaluates these operations in order.

Writing the network with matrices

Collect the hidden-unit computations into vectors:

$$\mathbf{a}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \qquad \mathbf{z} = h(\mathbf{a}^{(1)}).$$

Then compute the output:

$$\mathbf{a}^{(2)} = \mathbf{W}^{(2)}\mathbf{z} + \mathbf{b}^{(2)}, \qquad \mathbf{y} = f(\mathbf{a}^{(2)}).$$

For D inputs, M hidden units, and K outputs,

$$\mathbf{W}^{(1)} \in \mathbb{R}^{M \times D}, \qquad \mathbf{W}^{(2)} \in \mathbb{R}^{K \times M}.$$

The total number of weights and biases is

$$\boxed{M(D+1) + K(M+1).}$$



Representing biases as extra inputs

Introduce a constant input $x_0 = 1$:

$$a_j^{(1)} = \sum_{i=0}^{D} w_{ji}^{(1)} x_i, \qquad w_{j0}^{(1)} = b_j^{(1)}.$$

In vector notation,

$$\widetilde{\mathbf{x}} = \begin{pmatrix} 1 \\ \mathbf{x} \end{pmatrix}, \qquad \widetilde{\mathbf{W}}^{(1)} = \begin{pmatrix} \mathbf{b}^{(1)} & \mathbf{W}^{(1)} \end{pmatrix}.$$

Similarly, append $z_0 = 1$ to the hidden activations for the output layer. This is only a notation change. The constant nodes do not add learnable hidden features.

Hidden units cooperate to form a function

For one input and an identity output,

$$y(x) = b^{(2)} + \sum_{j=1}^{M} c_j h(a_j x + b_j).$$

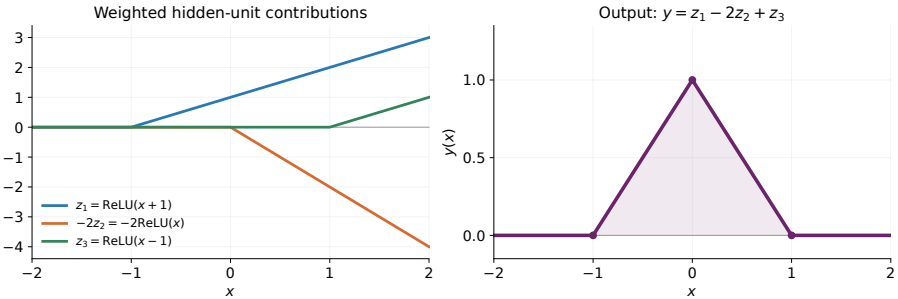
- a_j controls how quickly a feature changes and its orientation.
- b_j shifts its response along the input axis.
- c_j controls its contribution to the final function.

An individual feature can be simple, while their sum produces a complicated shape.

Hidden units cooperate to form a function

Three ReLU hidden units with output weights $1, -2, 1$ give

$$y(x) = \mathrm{ReLU}(x+1) - 2\,\mathrm{ReLU}(x) + \mathrm{ReLU}(x-1).$$



Each unit changes the slope. Their weighted sum forms a triangular peak, with height 1 at $x = 0$ and value 0 outside $[-1, 1]$.



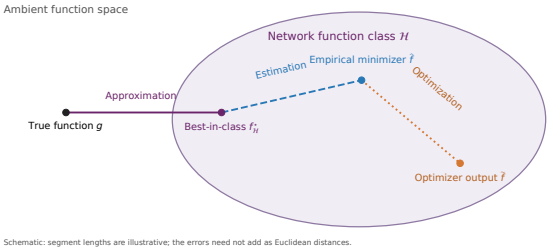
Representational capability and its limits

Under suitable conditions on the activation, a network with one hidden layer can approximate any continuous function on a compact domain to arbitrary accuracy, using enough hidden units.
Formally, for a continuous target g on compact $\mathcal{K}$ and any $\epsilon > 0$, there exists a network such that

$$\sup_{\mathbf{x} \in \mathcal{K}} |y(\mathbf{x}) - g(\mathbf{x})| < \epsilon.$$

- The result guarantees existence of a representation.
- It does not guarantee a small number of units.
- It does not guarantee that training finds that representation or that it generalizes from finite data.

Representation, data, and optimization



- **Approximation error:** How closely can the chosen network class represent the true function?
- **Estimation error:** How does using finite training data change the function we select?
- **Optimization error:** How closely does the training algorithm approach the minimum training loss?

The approximation theorem addresses only representational capability. It does not guarantee small estimation or optimization error.

Why hidden units need nonlinearity

If the hidden activation is the identity, then

$$\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}.$$

The output pre-activation becomes

$$\mathbf{a}^{(2)} = \underbrace{\mathbf{W}^{(2)}\mathbf{W}^{(1)}}_{\mathbf{W}_{\text{eff}}}\mathbf{x} + \underbrace{\mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}}_{\mathbf{b}_{\text{eff}}}.$$

Thus

$$\mathbf{y} = f(\mathbf{W}_{\text{eff}}\mathbf{x} + \mathbf{b}_{\text{eff}}).$$

It has the form of a single affine layer followed by the same output activation. Hidden linear layers add no nonlinear representation capability.

A narrow linear layer imposes a rank constraint

For linear hidden units,

$$\mathbf{W}_{\text{eff}} = \mathbf{W}^{(2)}\mathbf{W}^{(1)}, \qquad \operatorname{rank}(\mathbf{W}_{\text{eff}}) \leq M.$$

If $M < \min(D, K)$, the overall linear mapping cannot have full rank.

- The hidden layer compresses the input through M coordinates.
- Ignoring biases, the factorization uses $M(D + K)$ weights instead of DK.
- This reduces the parameter count when M is sufficiently small.

A linear autoencoder trained with squared reconstruction error has a connection to PCA under suitable conditions. An arbitrary linear bottleneck is not automatically PCA.

Smooth, bounded hidden activations

Logistic sigmoid

$$\sigma(a) = \frac{1}{1 + e^{-a}}, \quad 0 < \sigma(a) < 1.$$

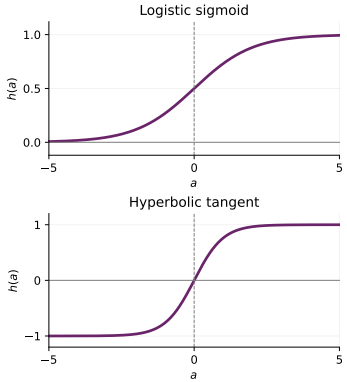
Hyperbolic tangent

$$\tanh(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}, \quad -1 < \tanh(a) < 1.$$

They are related by

$$\tanh(a) = 2\sigma(2a) - 1.$$

Both are smooth and saturate for large positive or negative inputs. Tanh permits signed feature values.



Saturation can weaken the learning signal

The derivatives are

$$\sigma'(a) = \sigma(a)\{1 - \sigma(a)\}, \qquad \tanh'(a) = 1 - \tanh^2(a).$$

For large $|a|$, these derivatives approach zero. For a single hidden unit $z = h(a)$, the chain rule gives

$$\frac{\partial E}{\partial a} = \frac{\partial E}{\partial z} h'(a).$$

Even if the loss is sensitive to z, a small $h'(a)$ can make it weakly sensitive to the incoming weights. Repeated multiplication by small derivatives can contribute to vanishing gradients in deeper networks.

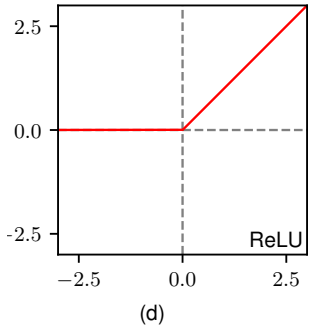
A simple piecewise-linear activation

The rectified linear unit (ReLU) is

$$h(a) = \max(0, a).$$

For $a \neq 0$,

$$h'(a) = \begin{cases} 0, & a < 0, \\ 1, & a > 0. \end{cases}$$



- Computationally inexpensive.
- No saturation for positive inputs.
- Zero gradient for negative inputs.

At $a = 0$, implementations use a derivative convention, commonly zero.

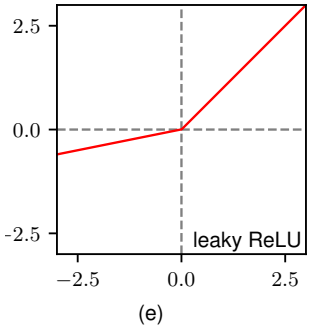
Allowing gradients for negative inputs

Leaky ReLU retains a negative-side slope:

$$h(a) = \begin{cases} \alpha a, & a < 0, \\ a, & a \geq 0, \end{cases} \qquad 0 < \alpha < 1.$$

$$h'(a) = \begin{cases} \alpha, & a < 0, \\ 1, & a > 0. \end{cases}$$

- Negative inputs still permit a local gradient.
- A parametric variant learns a slope α_j for each unit.

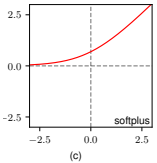


Smooth rectification and clipped activations

Softplus

$$h(a) = \log(1 + e^a), \qquad h'(a) = \sigma(a).$$

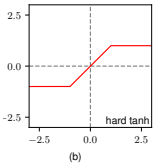
A smooth approximation to ReLU. For large positive a, the slope approaches 1.



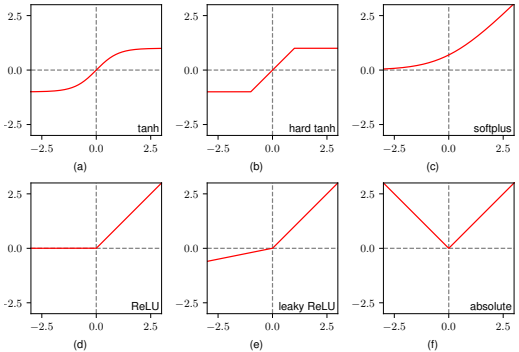
Hard tanh

$$h(a) = \max(-1, \min(1, a)).$$

Linear between -1 and 1, with zero slope outside. The derivative is undefined at the two clipping points.



Choosing hidden and output activations



Hidden layers

Activation shape affects feature learning and gradient propagation.

Output layer

The activation depends on the prediction task and probability model.

Hidden and output activations need not match.



Different weights can produce the same predictions

Consider a scalar output with tanh hidden units:

$$y(\mathbf{x}) = b + \sum_{j=1}^{M} c_j \tanh(\mathbf{a}_j^\top \mathbf{x} + d_j).$$

Since $\tanh(-a) = -\tanh(a)$, changing one hidden unit's parameters as

$$(\mathbf{a}_j, d_j, c_j) \longmapsto (-\mathbf{a}_j, -d_j, -c_j)$$

leaves its contribution unchanged:

$$(-c_j)\tanh(-\mathbf{a}_j^\top \mathbf{x} - d_j) = c_j \tanh(\mathbf{a}_j^\top \mathbf{x} + d_j).$$

The parameter vector changes, but the function represented by the network does not.

The order of hidden units is arbitrary

With two hidden units,

$$y = b + c_1 h(\mathbf{a}_1^\top \mathbf{x} + d_1) + c_2 h(\mathbf{a}_2^\top \mathbf{x} + d_2).$$

Swapping their incoming parameters and corresponding outgoing weights simply reorders the sum.

- With M units, there are $M!$ possible orderings.
- Tanh units additionally allow independent sign flips, giving a factor 2^M.
- These operations give up to $M!2^M$ equivalent parameter settings, with that many distinct settings in the generic case.

Training seeks a useful function. Its parameters need not be uniquely identifiable.
Other activations can have different symmetries. The tanh sign-flip argument does not apply directly to ReLU.



Table of Contents

- Limitations of Fixed Basis Functions
- Multilayer Networks
- Deep Networks
- Error Functions

Composing learned transformations

A deep network applies several learned transformations in succession:

$$\mathbf{z}^{(0)} = \mathbf{x}, \qquad \mathbf{z}^{(l)} = h^{(l)}\left(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}\right), \quad l = 1, \ldots, L.$$

$$\mathbf{z}^{(L)} = \mathbf{y}.$$

- Each layer receives the representation produced by the previous layer.
- We learn the weights and biases jointly for the desired task.
- Hidden activations introduce nonlinearity. The output activation depends on the task.

Here biases are explicit. The textbook includes them in augmented weight matrices.

Why add more layers?

A network with one hidden layer can approximate continuous functions on compact domains under suitable conditions.

- This is an existence result. It does not specify an efficient representation or guarantee successful learning.
- For some functions, a shallow network requires many more units than a deep network.
- Depth allows intermediate computations to be reused and combined.

Expressive efficiency

For certain ReLU architectures, the number of linear regions can grow exponentially with depth. The benefit depends on the architecture and function being represented.

Complex features built from simpler features

Hierarchical representations: later layers combine simpler features into more complex ones.

Consider recognizing a cat in an image.

Stage	Illustrative information
Input	Pixel intensities
Early layers	Edges and simple local patterns
Intermediate layers	Shapes and parts, such as eyes or whiskers
Later layers	Combinations useful for recognizing a cat

The architecture supports composition: later features combine earlier features. This is an inductive bias about how complex patterns can be constructed. It is an illustration, not a guarantee that every neuron has a named concept.



Encoding combinations of attributes

Distributed representations: a concept is encoded by a combination of activations. Suppose faces vary in three binary attributes: glasses, hat, and beard.

$$\mathbf{z} = (z_{\text{glasses}}, z_{\text{hat}}, z_{\text{beard}}).$$

Example	Attribute code
Glasses only	$(1, 0, 0)$
Hat and beard	$(0, 1, 1)$
All three	$(1, 1, 1)$
None	$(0, 0, 0)$

Three binary units can encode $2^3 = 8$ combinations. More generally, M binary units permit 2^M codes. A separate unit for each combination would require 2^M units.



A learned coordinate system

Representation learning discovers a transformation that makes a task easier.

Write a classifier as a learned feature extractor followed by a linear score:

$$\mathbf{z} = F_{\boldsymbol{\theta}}(\mathbf{x}), \qquad a = \mathbf{w}^{\top}\mathbf{z} + b.$$

- $F_{\boldsymbol{\theta}}$ transforms the original input into an embedding.
- The final classifier separates classes using a hyperplane in embedding space.
- That same boundary can be nonlinear in the original input space.

What the hidden layers learn

A representation that makes the prediction task easier for the final layer.

Perfect linear separation is not always possible, especially with noisy or overlapping classes.



How the learning approaches fit together

Common goal: learn a useful representation

$$\mathbf{z} = F_{\boldsymbol{\theta}}(\mathbf{x}).$$

The terminology describes different aspects of learning:

Question	Relevant approaches
What does the network learn?	Representation learning
What training signal is available?	Supervised or unsupervised learning
How is knowledge used across tasks?	Transfer, multitask, and meta-learning
How do we train an embedding?	Contrastive learning

These categories overlap. For example, contrastive pretraining can produce a representation that is later transferred to another task.

Learning without manual labels

Unsupervised representation learning uses data without manually assigned target labels. An image collection may be easy to obtain, while class labels require expert annotation. **Example: an autoencoder** uses the input itself as the reconstruction target:

$$\mathbf{z} = F_{\boldsymbol{\theta}}(\mathbf{x}), \qquad \widehat{\mathbf{x}} = G_{\boldsymbol{\psi}}(\mathbf{z}).$$

$$\min_{\boldsymbol{\theta},\boldsymbol{\psi}} \sum_{n} \|\widehat{\mathbf{x}}_n - \mathbf{x}_n\|^2.$$

A bottleneck can force compression. The learned representation may help another task, although reconstruction alone does not guarantee this.

Pretraining and a new prediction task

Transfer learning reuses knowledge learned on a source task to help a related target task.

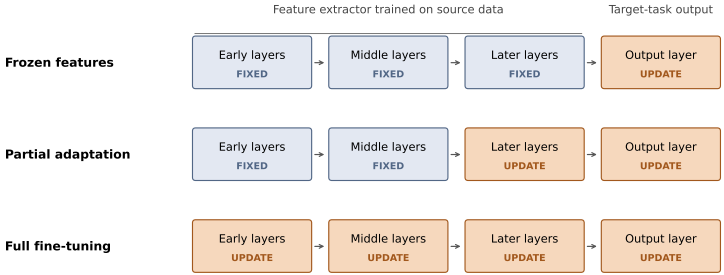
$$\mathbf{z} = F_{\boldsymbol{\theta}}(\mathbf{x}), \qquad \widehat{\mathbf{y}} = H_{\boldsymbol{\eta}}(\mathbf{z}).$$

- Source-task pretraining learns the parameters $\boldsymbol{\theta}$.
- Training on the new task learns the prediction head $\boldsymbol{\eta}$ and may also update $\boldsymbol{\theta}$.
- The source representation must contain information relevant to the new task.

Example: features learned from everyday object images can help train a skin-lesion classifier with fewer labelled images.

Freezing features or adapting the network

After training on source data, replace the output layer for the target task.



Fixed: reuse the pretrained weights without changing them. **Update:** learn from target-task data.

Partial adaptation can update a different number of later layers. Fine-tuning commonly uses a small learning rate for pretrained layers.

Sharing information across tasks

Multitask learning trains related tasks jointly, using shared features and task-specific heads:

$$\mathbf{z} = F_{\boldsymbol{\theta}}(\mathbf{x}), \qquad \widehat{\mathbf{y}}^{(k)} = H_{\boldsymbol{\eta}_k}(\mathbf{z}).$$

$$\min_{\boldsymbol{\theta}, \{\boldsymbol{\eta}_k\}} \sum_{k=1}^{K} \lambda_k \mathcal{L}_k(\boldsymbol{\theta}, \boldsymbol{\eta}_k).$$

Example: spam filters for different users share general email features but retain different user preferences.

- Joint learning pools information about related tasks.
- Shared parameters receive learning signals from multiple tasks.
- Benefits depend on how well the tasks fit a shared representation.



Adapting to previously unseen tasks

Meta-learning trains across tasks to learn how to adapt to a previously unseen task.

- The training procedure learns an initialization, representation, or update rule that supports adaptation.
- At evaluation, a new task provides a small set of labelled examples.
- The model uses those examples to adapt, then predicts on other examples from that task.

Few-shot learning: adaptation with very few labelled examples per class.

One-shot learning: the special case with one labelled example per class.

Few-shot describes the limited-data setting. Meta-learning is one approach to addressing it.

Reusing, sharing, and adapting knowledge

Approach	Training arrangement	Intended benefit
Transfer learning	Learn on source data, then train or adapt on a target task	Use existing knowledge when target data are limited
Multitask learning	Train a fixed collection of related tasks jointly	Share information among the known tasks
Meta-learning	Train across tasks, then adapt to a new task	Learn how to adapt with little task-specific data

Few-shot learning specifies that very few labelled examples per class are available. It can use pretrained features, fine-tuning, or meta-learning.



Learning by comparing examples

Contrastive learning learns an embedding using positive and negative comparisons.

For an anchor $\mathbf{x}$, choose a positive example $\mathbf{x}^+$ and negative examples $\mathbf{x}_1^-, \ldots, \mathbf{x}_N^-$.

$$\mathbf{z} = f_{\mathbf{w}}(\mathbf{x}), \qquad \mathbf{z}^+ = f_{\mathbf{w}}(\mathbf{x}^+), \qquad \mathbf{z}_n^- = f_{\mathbf{w}}(\mathbf{x}_n^-).$$

- Encourage the anchor and positive to have similar representations.
- Encourage the anchor to be less similar to the negatives.
- The choice of pairs specifies what similarity should mean.

Unlike an ordinary per-example prediction loss, this loss compares the anchor with other examples.

Selecting the positive among candidate examples

For unit-length embeddings, dot products measure cosine similarity. Let $s^+ = \mathbf{z}^\top \mathbf{z}^+$ and $s_n^- = \mathbf{z}^\top \mathbf{z}_n^-$. Define a softmax probability over the positive and negative candidates:

$$p_+ = \frac{\exp(s^+)}{\exp(s^+) + \sum_{n=1}^{N} \exp(s_n^-)}.$$

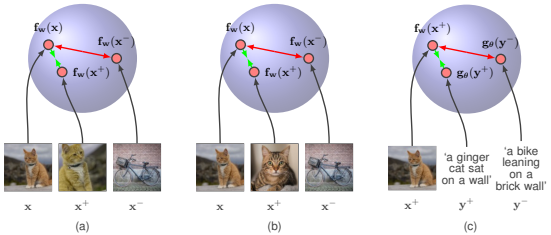
The textbook's InfoNCE loss is

$$\boxed{\mathcal{L} = -\log p_+.}$$

- Treat the positive as the correct candidate.
- Minimizing the loss rewards higher positive similarity relative to negative similarities.
- The objective has the same form as a classification cross-entropy loss.

Choosing positive and negative pairs

An **anchor** is the reference input. A **positive** should match it; a **negative** should not.



(a) Same image

Instance discrimination: match two augmented views of one photo.

(b) Same class

Supervised contrastive learning: match two different cat photos using class labels.

(c) Matching description

CLIP: match a cat photo with its caption, rather than a bicycle caption.

The objective rewards **greater similarity to the positive than to negatives**.

Images and text in a shared space

CLIP learns from matched image–caption pairs. Use separate encoders for the two modalities:

$$\mathbf{u}_i = f_{\mathbf{w}}(\mathbf{x}_i), \qquad \mathbf{v}_j = g_{\boldsymbol{\theta}}(\mathbf{c}_j), \qquad \|\mathbf{u}_i\| = \|\mathbf{v}_j\| = 1.$$

For a batch of B matching image–caption pairs, define

$$S_{ij} = \mathbf{u}_i^\top \mathbf{v}_j.$$

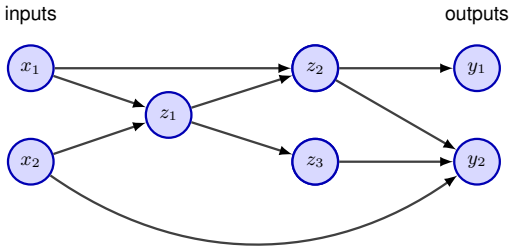
Matching pairs lie on the diagonal of S.

- Each image should select its matching caption among the candidate captions.
- Each caption should select its matching image among the candidate images.
- Training uses both directions of comparison.

Connections beyond sequential layers

Each unit combines its directly connected inputs:

$$z_k = h_k\left(\sum_{j \in A(k)} w_{kj} z_j + b_k\right).$$



$A(k)$ lists the nodes with arrows entering node k.

- Connections can branch or skip layers.
- There are no directed cycles.
- Biases are omitted from the diagram.

For example, y_2 receives three inputs, including a direct connection from x_2:

$$y_2 = f_2(w_{y_2,z_2} z_2 + w_{y_2,z_3} z_3 + w_{y_2,x_2} x_2 + b_{y_2}).$$



Table of Contents

- Limitations of Fixed Basis Functions
- Multilayer Networks
- Deep Networks
- Error Functions

A probability model determines the loss

The output specifies a conditional distribution for the target. For independent training observations, minimize

$$E(\mathbf{w}) = -\sum_{n=1}^{N} \log p(t_n \mid \mathbf{x}_n, \mathbf{w}).$$

Target	Probability model	Output activation
Real value	Gaussian	Identity
Binary label	Bernoulli	Sigmoid
One of K classes	Categorical	Softmax

The hidden layers learn features. The output activation and loss must match what the target represents.

Regression: predict the mean of a Gaussian

For a continuous target,

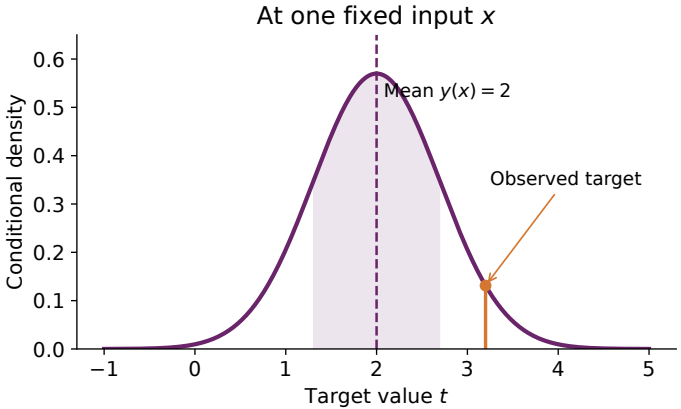
$$t \mid \mathbf{x} \sim N(y(\mathbf{x}, \mathbf{w}), \sigma^2).$$

- Use identity output $y = a$.
- y is the predicted mean.
- σ^2 describes residual variation.

For one observation,

$$-\log p(t \mid \mathbf{x}) = \frac{(t-y)^2}{2\sigma^2} + \text{constant},$$

when σ^2 is fixed.



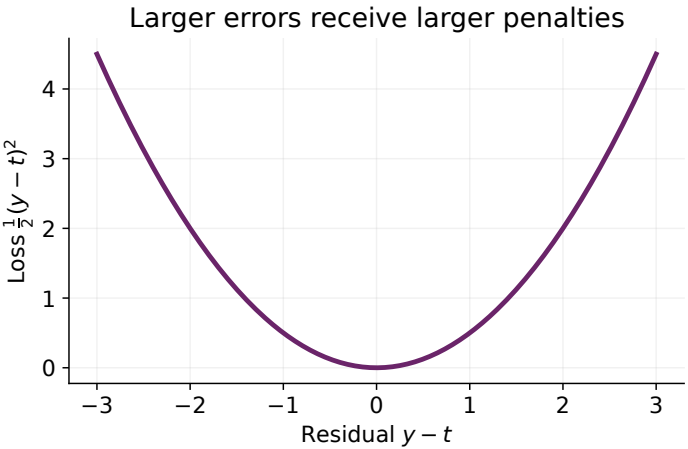
The input is fixed: the curve describes possible target values. Fitting the Gaussian mean therefore minimizes squared error.

Regression: errors and residual variation

For fixed Gaussian variance, minimize

$$E_{\mathrm{SE}} = \frac{1}{2}\sum_n (y_n - t_n)^2.$$

- Positive and negative residuals are penalized equally.
- Doubling a residual quadruples its loss.



After fitting the mean, estimate the variance by maximum likelihood:

$$\widehat{\sigma}^2 = \frac{1}{N}\sum_n (y_n - t_n)^2.$$

For K independent Gaussian targets with shared variance, sum over coordinates and use NK in the variance denominator.

Binary classification: a probability for each label

For $t \in \{0, 1\}$, use

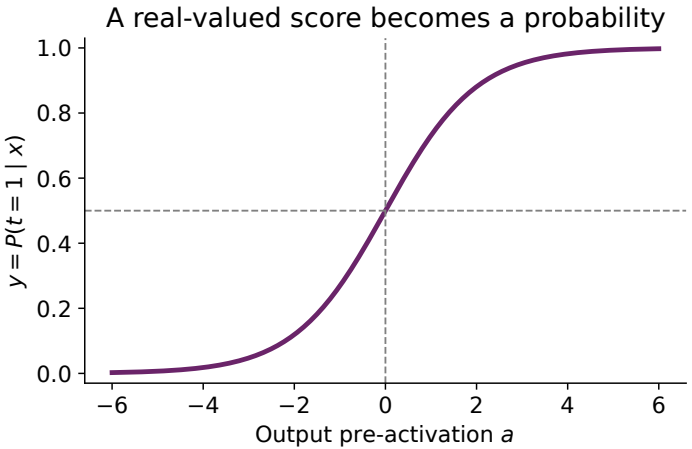
$$y = \sigma(a) = \frac{1}{1 + e^{-a}}.$$

Then

$$P(t = 1 \mid \mathbf{x}) = y,$$
$$P(t = 0 \mid \mathbf{x}) = 1 - y.$$

The Bernoulli model is

$$p(t \mid \mathbf{x}) = y^t (1 - y)^{1-t}.$$



Its negative log-likelihood gives binary cross-entropy:

$$E = -\sum_n [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

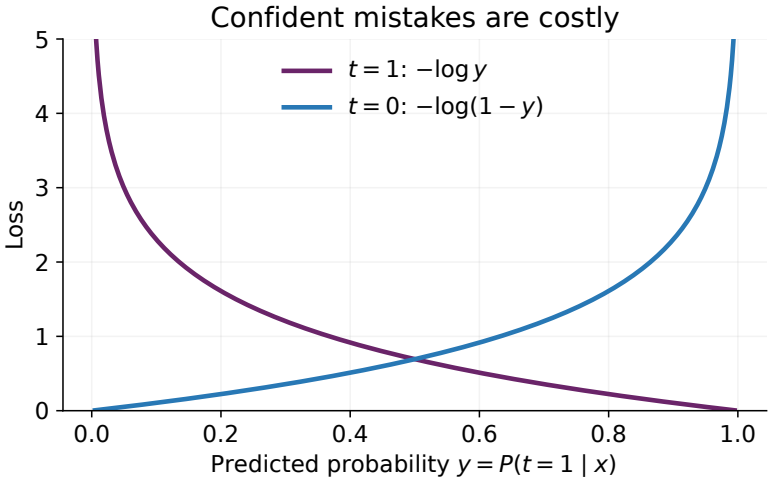
Cross-entropy penalizes confident mistakes

For an observed positive label $t = 1$,

$$\ell = -\log y.$$

Predicted y	Loss
0.9	0.105
0.5	0.693
0.1	2.303
0.01	4.605

A confident wrong prediction gets a much larger penalty than an uncertain one.



One of several classes: softmax and cross-entropy

For mutually exclusive classes,

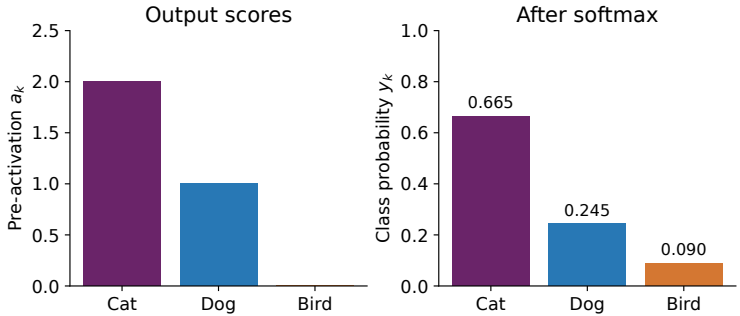
$$y_k = \frac{e^{a_k}}{\sum_j e^{a_j}}, \qquad \sum_k y_k = 1.$$

Use a one-hot target. For a cat,

$$\mathbf{t} = (1, 0, 0).$$

The categorical model gives

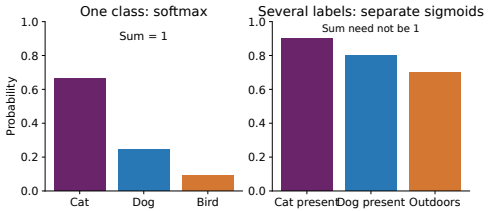
$$p(\mathbf{t} \mid \mathbf{x}) = \prod_k y_k^{t_k}.$$



$$E = -\sum_{n,k} t_{nk} \log y_{nk}.$$

For a cat target, only $-\log y_{\text{cat}}$ contributes. In this example, $-\log(0.665) \approx 0.408$.

Several labels can be true simultaneously



Left: exactly one class applies, so use softmax.

Right: cat, dog, and outdoors can all be true, so use separate sigmoids.

Assuming labels are conditionally independent, sum their binary cross-entropies:

$$E = -\sum_{n,k} [t_{nk} \log y_{nk} + (1 - t_{nk}) \log(1 - y_{nk})].$$

Key distinction: separate sigmoid probabilities need not sum to 1.



Output and loss pairings to remember

Task	Output	Loss
Regression	Identity	Half squared error
Binary classification	Sigmoid	Binary cross-entropy
Several binary labels	Separate sigmoids	Summed binary cross-entropy
One of K classes	Softmax	Multiclass cross-entropy

For one observation, these pairings share the useful output gradient

$$\boxed{\frac{\partial \ell}{\partial a_k} = y_k - t_k.}$$

Regression here uses half squared error with the fixed Gaussian variance scale removed. Batch averaging introduces a corresponding factor.

Check: If a photo can contain both a cat and a bicycle, why would a two-class softmax be inappropriate?